Del cliente al evento: tres versiones

main 513fa74 → tus commits 57250e7 / 924cf3b / ac868cf → Felipe 1dde0e4

Ruta común: el cliente llama al Servicio de Solicitudes. Solicitudes llama por HTTP al Servicio de Notificaciones. Cada BD pertenece a un servicio concreto; el cambio principal es cuándo se hace el POST y qué queda guardado si falla.

1. La diferencia en una mirada

Estado	BD de Solicitudes	BD de Notificaciones	HTTP y recuperación
main	Única BD del sistema; guarda la solicitud.	No existía; notificación en memoria.	1 POST inmediato. Si falla, solo log.
Tus cambios	db-solicitudes; guarda la solicitud.	db-notificaciones guarda IDs; notificación en memoria.	Hasta 3 POST inmediatos. Si fallan, solo log.
Felipe	db-solicitudes guarda solicitud + evento pendiente.	Igual que tu etapa.	Relevador: 3 POST por ciclo; si falla, conserva evento.

2. Qué aportó cada persona

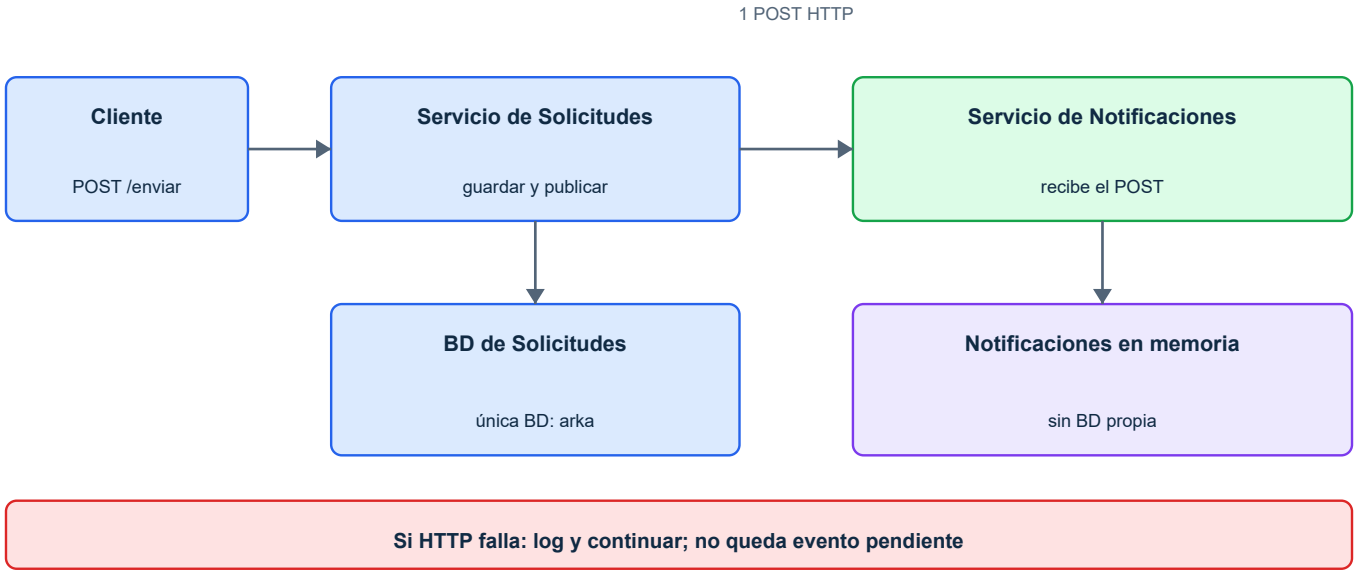
Commits	Aporte
Tú · 57250e7, 924cf3b, ac868cf	Separación de bases, eventoId, deduplicación en Notificaciones, publicador con tres intentos y limpieza del laboratorio.
Felipe · 1dde0e4	Outbox duradero en Solicitudes, transacción inicial, relevador periódico y ajuste de pruebas.

Outbox y deduplicación son piezas distintas: Solicitudes guarda lo pendiente de enviar en su tabla evento; Notificaciones guarda los ID ya recibidos en su tabla evento_procesado.

3. Escenario guía

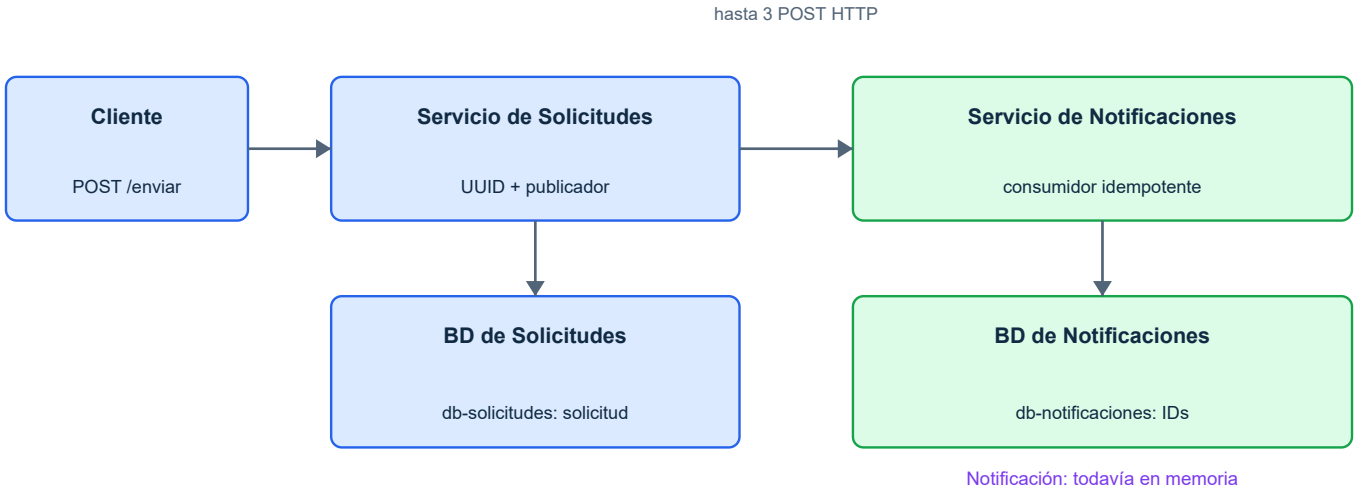
Supongamos que el cliente envía la solicitud INC-002 mientras Notificaciones no responde. Las páginas siguientes muestran qué hace cada versión y si ese aviso puede recuperarse.

4. En main: una sola base de datos



Secuencia: cliente → Solicitudes → su BD → POST inmediato a Notificaciones → memoria. La tabla evento existía en el esquema de Solicitudes, pero enviar() no la usaba. Si falla el único POST, PublicadorRest captura la excepción y la solicitud queda enviada sin aviso recuperable.

5. Con tus cambios: dos bases y tres intentos

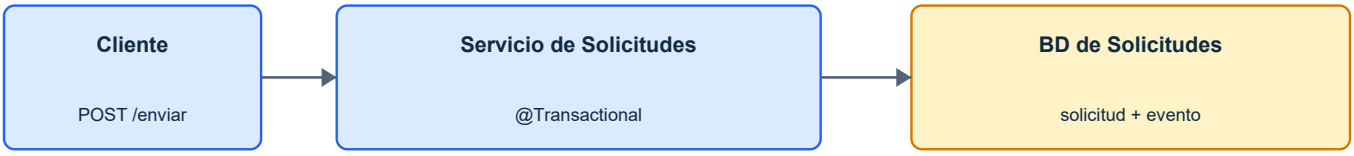


Secuencia: cliente → Solicitudes → db-solicitudes → hasta tres POST a Notificaciones. El mismo eventoId se usa en los tres intentos. Notificaciones consulta su db-notificaciones para ignorar un ID repetido, guarda allí el ID nuevo y crea la notificación en memoria.

Tu etapa añadió identificación, deduplicación y tolerancia a fallos breves de HTTP. Todavía no había un evento saliente persistido para intentar de nuevo tras agotar los tres POST.

6. Con Felipe: primero se guarda; después se publica

FASE 1 | petición del cliente

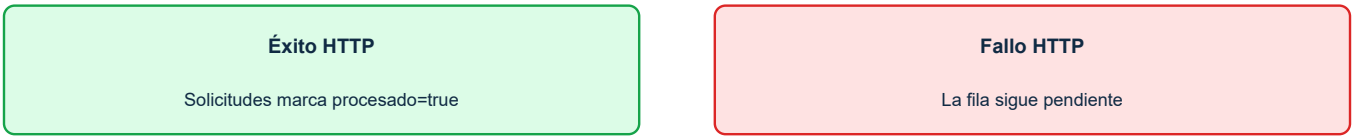


Responde ENVIADA sin esperar el HTTP

FASE 2 | publicación posterior



Notificación en memoria



Siguiente ciclo: mismo eventoId

El relevador pertenece a Solicitudes. Lee procesado=false de la BD de Solicitudes, envía el mismo eventoId por HTTP y, si el POST termina bien, marca procesado=true. Notificaciones conserva tu deduplicación en su BD y su notificación continúa en memoria.

7. Ciclo de reintentos

Paso	Qué hace el código
Pendiente	evento.procesado=false en db-solicitudes.
Publicación	Hasta tres POST: esperas de 500 ms y 1000 ms antes de los siguientes intentos.
Éxito	El relevador marca procesado=true en db-solicitudes.
Fallo	Lanza PublicacionFallidaException; la fila sigue pendiente para otro ciclo.
Ciclo nuevo	fixedDelay=5000: empieza unos 5 s después de terminar el anterior.

La comunicación entre microservicios sigue siendo HTTP/REST. El outbox cambia cuándo y desde dónde se hace la llamada; no reemplaza el transporte.

8. Comparación técnica lado a lado

Tema	main	Tus cambios	Felipe
enviar()	Guarda y hace 1 POST.	Guarda y hace hasta 3 POST.	Guarda estado + outbox; HTTP después.
eventoId	No existe.	UUID estable en esa llamada.	UUID persistido entre ciclos.
Si HTTP falla	Log; sin recuperación.	Log tras 3 intentos.	Evento pendiente; siguiente ciclo.
BD de Notificaciones	No existe.	Solo ID procesados.	Sin cambios respecto a tu etapa.
Notificación	En memoria.	En memoria.	En memoria.
tipo del mensaje	Tipo de negocio.	Tipo de negocio.	Literal SolicitudEnviada.
Respuesta al cliente	Espera el POST.	Espera hasta 3 POST.	No espera entrega HTTP.

9. Tablas y contratos actuales

Propietario	Dato	Finalidad
Solicitudes	solicitud	Estado de negocio, incluida ENVIADA.
Solicitudes	evento: evento_id, procesado	Outbox saliente y seguimiento de publicación.
Notificaciones	evento_procesado: evento_id	Identifica eventos que ya fueron recibidos.
HTTP	POST /eventos/solicitud-enviada	Mensaje con eventoId, id y tipo.

10. Límites del estado actual

Notificaciones guarda el objeto en memoria aunque registre el ID en PostgreSQL: al reiniciar puede perderse la notificación y conservarse el ID. Varias réplicas del relevador pueden publicar la misma fila porque no hay reclamación o bloqueo visible. El relevador tampoco registra la causa de los errores capturados.

El esquema de Felipe usa CREATE TABLE IF NOT EXISTS: una tabla evento creada antes necesita migración para añadir evento_id y procesado. Además, Felipe publica el literal SolicitudEnviada en el campo tipo; antes se enviaba el tipo de negocio de la solicitud.

Fuente: código en main 513fa74, ac868cf y 1dde0e4. Descripción del código; no certifica pruebas de integración ni despliegue.